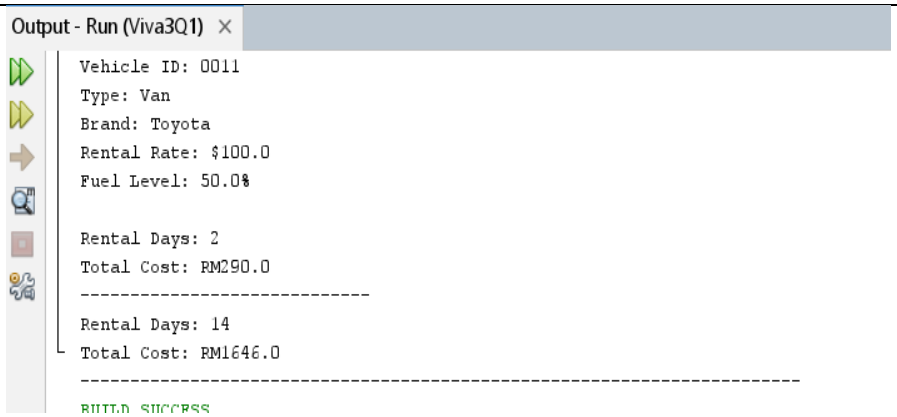
LAB REPORT VIVA 3

Team 5: Business Buzz

No.	Matric No.	Name
1.	24067901	Shaif Ahmad Bin Tufail Ahmad
2.	23121478	Tanvir Hossain
3.	24004539	Vosshindran A/L Thiyagaraja
4.	24064422	Sifat Ul Islam Babor
5.	23002044	Karl Xavier Franky
6.	23103329	Osman Mohammedelamin Elsharief Ahmed

Question 1

Section	Description
1. Problem	Design a `Vehicle` class for a vehicle rental system that: • Manages vehicles of various types (`Car`, `Van`, `Motorcycle`). • Incorporates tiered pricing models based on vehicle type and rental duration. • Includes additional rules like fuel charges. • Provides constructors, accessors, mutators, and methods to calculate rental cost and display information.
2. Solution	**Class Structure** • **Constructors**: An empty constructor initializes default values, and another accepts parameters. • **Accessor/Mutator Methods**: For all fields (`vehicleID`, `type`, `brand`, `rentalRate`, `fuelLevel`). • **Methods**: `calculateRentalRate`: Adjusts rates for vans (+20%) and motorcycles (-25%). `calculateRentalCost(int days)`: Calculates total cost with discounts: - 5% for rentals of 7–14 days. -10% for rentals longer than 14 days. `calculateFuelCharge`: Adds a RM50 penalty if the fuel level is below 50%. `toString`: Summarizes the vehicle's rental details, including final cost.
3. Sample Input & Output	

4. Source Code	//vehicle class
	public class Vehicle {
	private String vehicleID;
	private String type;

```
    private String brand;
    private double rentalRate;
    private double fuelLevel;
    //default
    public Vehicle(){
      this.vehicleID= "0000";
       this.type= "Car";
        this.brand= "Myvi";
         this.rentalRate= 100.0;
          this.fuelLevel= 100.0;
    }
     public Vehicle(String vehicleID, String type, String brand, double
rentalRate,  double fuelLevel){
      this.vehicleID = vehicleID;
      this.type = type;
      this.brand = brand;
      this.rentalRate = rentalRate;
      this.fuelLevel = fuelLevel;
    }

     public String getVehicleID() {
      return vehicleID;
    }
    public void setVehicleID(String vehicleID) {
      this.vehicleID = vehicleID;
    }
    public String getType() {
      return type;
    }
    public void setType(String type) {
      this.type = type;
    }
    public String getBrand() {
      return brand;
    }
    public void setBrand(String brand) {
      this.brand = brand;
    }
    public double getRentalRate() {
      return rentalRate;
    }
```

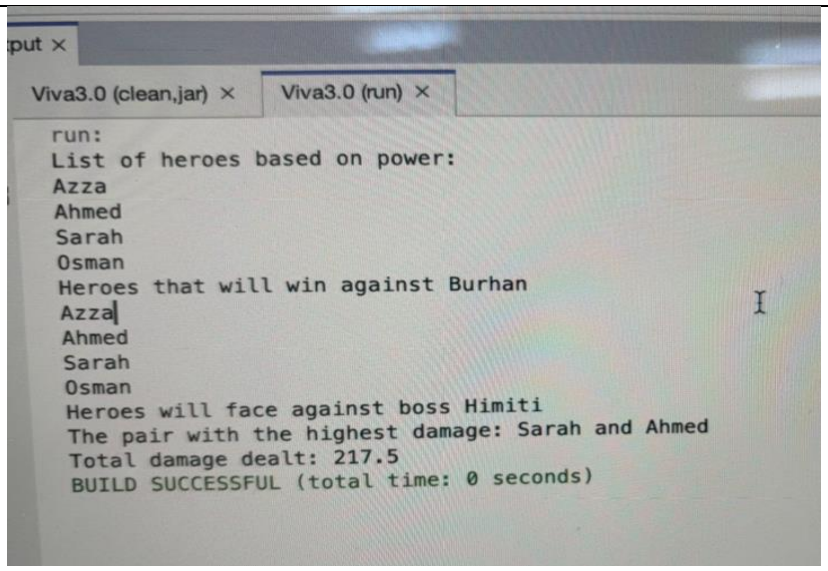
```
public void setRentalRate(double rentalRate) {
   this.rentalRate = rentalRate;
}
public double getFuelLevel() {
   return fuelLevel;
}
 public void setFuelLevel(double fuelLevel) {
   this.fuelLevel = fuelLevel;
}
 public double calculateRentalRate() {
   double adjustedRate = rentalRate;
   //checks eventhough if its van or Van
   if (type.equalsIgnoreCase("Van")) {
      adjustedRate *= 1.2; // 20% more expensive
   } else if (type.equalsIgnoreCase("Motorcycle")) {
      adjustedRate *= 0.75; // 25% less expensive
   }
   return adjustedRate;
}
 public double calculateRentalCost(int days) {
   double rate = calculateRentalRate();
   double totalCost = rate * days;

   // Apply discounts based on rental duration
   if (days >= 7 && days <= 14) {
      totalCost *= 0.95; // 5% discount
   } else if (days > 14) {
      totalCost *= 0.90; // 10% discount
   }

   // Add fuel charge if applicable
   totalCost += calculateFuelCharge();
   return totalCost;
}

// Method to calculate fuel charges
public double calculateFuelCharge() {
   // RM50 penalty if fuel < 50%
  if (fuelLevel <= 50.0) {
   return 50.0;
}
```

```
    return 0.0; // No penalty
}



    @Override
    public String toString() {
        return "Vehicle ID: " + vehicleID +
                "\nType: " + type +
                "\nBrand: " + brand +
                "\nRental Rate: $" + rentalRate +
                "\nFuel Level: " + fuelLevel + "%" +
                "\n";
    }
}

//main
public class Main {
    public static void main(String[] args) {
        // Create a vehicle (Van) with fuel level below 50%
        Vehicle van = new Vehicle("0011", "Van", "Toyota", 100.0, 50.0);

        // Display vehicle details
        System.out.println(van);

        // Calculate and display rental cost for 2 days
        int rentalDays = 2;
        System.out.println("Rental Days: " + rentalDays);
        System.out.println("Total Cost: RM" +
van.calculateRentalCost(rentalDays));

        System.out.println("----------------------------");

        // Calculate and display rental cost for 14 days
        rentalDays = 14;
        System.out.println("Rental Days: " + rentalDays);
        System.out.println("Total Cost: RM" +
van.calculateRentalCost(rentalDays));
    }
}
```

Question 2

Section	Description
1. Problem	Giving the heroes the necessary level of stratgey to defeat monsters following certain requierments like creating 3 classes and two speical methods

2. Solution	• **Hero Class**:
	• Stores the hero's name, element, and power.
	• The calculateDamage method computes damage dealt to a monster using the formula provided.
	• **Monster Class**:
	• Stores the monster's name, elemental resistances, and health points.
	• **HeroParty Class**:
	• Manages a list of heroes.
	• Provides methods:
	  ○ sortHeroesByPower: Sorts heroes by their power and prints them.
	  ○ battleWinners: Prints heroes that can defeat a given monster.
	  ○ battleBoss: Determines the pair of heroes that deal the most damage to a boss, including buffs.
	• **Main Class**:
	• Creates heroes and monsters.
	• Demonstrates sorting, evaluating heroes against a monster, and determining the best hero pair for a boss battle.

3. Sample Input & Output	

4. Source Code	

```
import java.util.*;

class Hero {
   String name;
   String element; // Pyro, Cryo, Hydro, Electro
   double power;

   Hero(String name, String element, double power) {
      this.name = name;
      this.element = element;
      this.power = power;
   }

   public double calculateDamage(Monster monster) {
      double resistance = switch (this.element) {
         case "Pyro" -> monster.pyroRes;
         case "Hydro" -> monster.hydroRes;
         case "Electro" -> monster.electroRes;
         case "Cryo" -> monster.cryoRes;
         default -> 0;
      };
      return this.power * ((100 - resistance) / 100);
   }
}

class Monster {
   String name;
   double pyroRes;
   double hydroRes;
   double electroRes;
   double cryoRes;
   double healthPoints;

   Monster(String name, double pyroRes, double hydroRes, double
electroRes, double cryoRes, double healthPoints) {
      this.name = name;
      this.pyroRes = pyroRes;
      this.hydroRes = hydroRes;
      this.electroRes = electroRes;
      this.cryoRes = cryoRes;
      this.healthPoints = healthPoints;
```

```
    }
}

class HeroParty {
    List<Hero> heroes;

    HeroParty() {
        heroes = new ArrayList<>();
    }

    public void addHero(Hero hero) {
        heroes.add(hero);
    }

    public void sortHeroesByPower() {
        heroes.sort(Comparator.comparingDouble(h -> h.power));
        System.out.println("List of heroes based on power:");
        for (Hero hero : heroes) {
            System.out.println(hero.name);
        }
    }

    public void battleWinners(Monster enemy) {
        System.out.println("Heroes that will win against " + enemy.name);
        for (Hero hero : heroes) {
            double damage = hero.calculateDamage(enemy);
            if (damage >= enemy.healthPoints) {
                System.out.println(hero.name);
            }
        }
    }

    public void battleBoss(Monster boss) {
        System.out.println("Heroes will face against boss " + boss.name);

        double maxDamage = 0;
        String bestPair = "";
        for (int i = 0; i < heroes.size(); i++) {
            for (int j = 0; j < heroes.size(); j++) {
                if (i != j) {
                    Hero hero1 = heroes.get(i);
```

```
            Hero hero2 = heroes.get(j);

            double damage1 = hero1.calculateDamage(boss);
            double damage2 = hero2.calculateDamage(boss);

            // Apply buffs based on element combination
            if ((hero1.element.equals("Pyro") &&
hero2.element.equals("Hydro")) ||
              (hero1.element.equals("Hydro") &&
hero2.element.equals("Pyro"))) {
              damage1 *= 1.5;
            } else if ((hero1.element.equals("Pyro") &&
hero2.element.equals("Cryo")) ||
                  (hero1.element.equals("Cryo") &&
hero2.element.equals("Pyro"))) {
              damage1 *= 2.0;
            } else if ((hero1.element.equals("Pyro") &&
hero2.element.equals("Electro")) ||
                  (hero1.element.equals("Electro") &&
hero2.element.equals("Pyro"))) {
              damage1 += new Random().nextInt(51) + 50;
            } else if ((hero1.element.equals("Hydro") &&
hero2.element.equals("Electro")) ||
                  (hero1.element.equals("Electro") &&
hero2.element.equals("Hydro"))) {
              damage1 += (new Random().nextInt(20) + 1) * 5;
            } else if ((hero1.element.equals("Electro") &&
hero2.element.equals("Cryo")) ||
                  (hero1.element.equals("Cryo") &&
hero2.element.equals("Electro"))) {
              damage2 *= 0.9; // Reduce resistance by 10%
            }

            double totalDamage = damage1 + damage2;

            if (totalDamage > maxDamage) {
              maxDamage = totalDamage;
              bestPair = hero1.name + " and " + hero2.name;
            }
          }
        }
```

```
        }

        System.out.println("The pair with the highest damage: " +
bestPair);
        System.out.println("Total damage dealt: " + maxDamage);
    }
}

public class GenshinImpact {
    public static void main(String[] args) {
        HeroParty party = new HeroParty();
        party.addHero(new Hero("Sarah", "Pyro", 100));
        party.addHero(new Hero("Osman", "Hydro", 120));
        party.addHero(new Hero("Azza", "Electro", 80));
        party.addHero(new Hero("Ahmed", "Cryo", 90));

        Monster normie = new Monster("Burhan", 20, 30, 10, 40, 50);
        Monster boss = new Monster("Himiti", 25, 25, 25, 25, 500);

        party.sortHeroesByPower();
        party.battleWinners(normie);
        party.battleBoss(boss);
    }
}
```

Question 3

Section	Description
1. Problem	Design and implement a **Room Rental Management System** in Java to manage room details and calculate rental costs. The system should meet the following requirements: 1. Maintain **room details** (e.g., room number, type, and rental price) while tracking the total number of rooms created. 2. Support methods to calculate rental costs, both with and without a discount, using method overloading. 3. Ensure proper **encapsulation** through getter and setter methods. 4. Display room details and rental costs in a structured and user-friendly format using static methods. 5. Validate input to prevent invalid data, such as negative values for days or discounts.
2. Solution	The **Room Rental Management System** was implemented in Java with the following features: 1. **Room Class Implementation:** o Contains private attributes for room details (roomNumber, roomType, and rentalPrice) and a static variable totalRooms to track the total number of rooms created. o Provides constructors, getter, and setter methods to ensure proper encapsulation. o Includes overloaded calculateRent methods to compute rental costs: ▪ **Without discount:** Calculates the total rent for a given number of days. ▪ **With discount:** Computes the total rent after applying a fixed nightly discount. 2. **Static Methods:** o displayRoomDetails outputs the room details, including the room number, type, rental price, and total rooms created. o displayRent formats and displays the rental costs for a given number of days, with and without discounts. 3. **Validation:** o Throws an IllegalArgumentException if invalid inputs, such as negative days or discounts, are provided. 4. **Main Method:** o Instantiates three Room objects representing different room types (Single, Double, Suite). o Calls displayRoomDetails to print room information. o Computes and displays the rental costs for 5 days, with and without a $10 nightly discount, using the calculateRent methods.

	○ Outputs are formatted with two decimal places for monetary values, ensuring clarity and professionalism.

3. Sample Input & Output	`Room Number: 101`
	`Room Type: Single`
	`Rental Price: $200.00`
	`Total rooms: 3`
	`Total Rent for 5 days: $1000.00`
	`Total rent for 5 days with $10.00 discount per night: $950.00`
	
	`Room Number: 205`
	`Room Type: Double`
	`Rental Price: $350.00`
	`Total rooms: 3`
	`Total Rent for 5 days: $1750.00`
	`Total rent for 5 days with $10.00 discount per night: $1700.00`
	
	`Room Number: 311`
	`Room Type: Suite`
	`Rental Price: $700.00`
	`Total rooms: 3`
	`Total Rent for 5 days: $3500.00`
	`Total rent for 5 days with $10.00 discount per night: $3450.00`

4. Source Code	`package com.mycompany.viva3;`
	
	`class Room{`
	`  private String roomNumber;`
	`  private String roomType;`
	`  private double rentalPrice;`
	`  private static int totalRooms;`
	
	`  public Room(String roomNumber, String roomType, double rentalPrice){`
	`  this.roomNumber = roomNumber;`
	`  this.roomType = roomType;`
	`  this.rentalPrice = rentalPrice;`
	`  totalRooms++;`
	`  }`
	
	`  public String getRoomNumber(){`
	`  return this.roomNumber ;`
	`  }`
	
	`  public String getRoomType(){`
	`  return this.roomType;`
	`  }`
	
	`  public double getRentalPrice(){`
	`  return this.rentalPrice;`
	`  }`

```
    public void setRoomNumber(String roomNumber){
        this.roomNumber = roomNumber;
    }

    public void setRoomType(String roomType){
        this.roomType = roomType;
    }

    public void setRentalPrice(double rentalPrice){
        this.rentalPrice = rentalPrice;
    }

    public double calculateRent(int days){
        return days * this.rentalPrice;
    }

    public double calculateRent(int days, double discount){
        return (this.rentalPrice - discount) * days;
    }

    public static Integer getTotalRooms(){
        return totalRooms;
    }

    public static void displayRoomDetails(Room room){
        System.out.println("Room Number: " + room.getRoomNumber());
        System.out.println("Room Type: " + room.getRoomType());
        System.out.printf("Rental Price: $%.2f\n", room.getRentalPrice());
        System.out.println("Total rooms: "+ totalRooms);
    }
}



public class Viva3 {

    public static void displayRent(int days, double rent, double discount,
double discountedRent){
        System.out.printf("Total Rent for %d days: $%.2f\n", days, rent);
```

```
        System.out.printf("Total rent for %d days with $%.2f discount per
night: $%.2f\n", days, discount, discountedRent);
    }

    public static void main (String [] args){

        Room r1 = new Room ("101", "Single", 200.00);
        Room r2 = new Room ("205", "Double", 350.00);
        Room r3 = new Room ("311", "Suite", 700.00);

        int days = 5;
        double discount = 10.0;

        Room.displayRoomDetails(r1);
        displayRent(days, r1.calculateRent(days), discount,
r1.calculateRent(days, discount));
        System.out.println();
        Room.displayRoomDetails(r2);
        displayRent(days, r2.calculateRent(days), discount,
r2.calculateRent(days, discount));
        System.out.println();
        Room.displayRoomDetails(r3);
        displayRent(days, r3.calculateRent(days), discount,
r3.calculateRent(days, discount));
        System.out.println();
    }
}
```

Question 4

Section	Description
1. Problem	You are tasked with creating a simplified text-based adventure game in Java. The game has four primary entities: Player, Enemy, Item and Black Magic. Implement the following classes to manage these entities and create basic game interactions
2. Solution	1. Create a Player object with a name of your choice. 2. Create an Enemy object with a type of your choice. 3. Create a few Item objects like a "Health Potion" that restores health and a "Sword" that increases attack power. 4. The player and enemy take turns attacking each other using the attack methods. (written in combat log that will be generated by to String method) 5. The player can use items to regain health or increase attack power during their turn. 6. The game ends when either the player or the enemy is defeated.

3. Sample Input & Output	`Welcome to Skibidi Island!`
	`Enter your name: babor`
	`babor encountered an enemy! It is a goblin`
	
	`Entering battle with a goblin...`
	
	`It's your turn!`
	
	`Choose your action:`
	`1. Attack`
	`2. Use Items`
	`3. Run Away`
	`4. View Combat Log`
	`Enter your choice: 1`
	`babor attacks the goblin for 10 damage!`
	`The goblin attacks you for 5 damage!`
	
	`Choose your action:`
	`1. Attack`
	`2. Use Items`
	`3. Run Away`
	`4. View Combat Log`
	`Enter your choice: 1`
	`babor attacks the goblin for 10 damage!`
	`The goblin used Magic Orb! Its attack power is now greatly increased.`
	
	`Choose your action:`
	`1. Attack`
	`2. Use Items`
	`3. Run Away`

4. Source Code	Main Class:
	
	import java.util.*;
	
	public class Main {
	
	  public static void main(String[] args) {
	
	  Scanner input = new Scanner(System.in);
	
	  Random rand = new Random();
	
	  System.out.println("Welcome to Skibidi Island!");
	
	  System.out.print("Enter your name: ");
	
	  String playerName = input.nextLine();
	
	  Player player = new Player(playerName);
	
	  Item healthPotion = new Item("Health Potion", 20);
	
	  Item sword = new Item("Sword", 15);
	
	  boolean retry = true;
	
	  while (retry) {
	
	  System.out.print(playerName + " encountered an enemy! It is a
	");

```
        String enemyType = input.nextLine();

        Enemy enemy = new Enemy(enemyType);



        BlackMagic poison = new BlackMagic("Poison", 10);

        BlackMagic magicOrb = new BlackMagic("Magic Orb", 0);



        System.out.println("\nEntering battle with a " + enemyType +
"...");

        System.out.println("\nIt's your turn!");



        boolean isBattleOn = true;



        while (isBattleOn) {

            System.out.println("\nChoose your action:");

            System.out.println("1. Attack");

            System.out.println("2. Use Items");

            System.out.println("3. Run Away");

            System.out.println("4. View Combat Log");

            System.out.print("Enter your choice: ");

            int choice = input.nextInt();

            input.nextLine();
```

```
            switch (choice) {

                case 1:

                    int playerDamage = player.attack(enemy);

                    System.out.println(playerName + " attacks the " +
enemyType + " for " + playerDamage + " damage!");



                    if (!enemy.isAlive()) {

                        System.out.println("\nYou defeated the " + enemyType +
"!");

                        System.out.print("\nContinue adventuring? (Y for yes,
Type anything for no): ");

                        String ret = input.nextLine();

                        if (ret.equalsIgnoreCase("Y")) {

                            retry = true;

                            isBattleOn = false;

                        } else {

                            retry = false;

                            isBattleOn = false;

                        }

                    }
```

```
                if (enemy.isAlive()) {

                    if (rand.nextBoolean()) {

                        if (rand.nextBoolean()) {

                            poison.use(player, enemy);

                            System.out.println("The " + enemyType + " used
Poison! You took damage and your attack power is reduced.");

                        } else {

                            magicOrb.use(player, enemy);

                            System.out.println("The " + enemyType + " used
Magic Orb! Its attack power is now greatly increased.");

                        }

                    } else {

                        enemy.attack(player);

                        System.out.println("The " + enemyType + " attacks you
for " + enemy.getAttackPower() + " damage!");

                    }

                }



                if (!player.isAlive()) {

                    System.out.println("\nYou were defeated by the " +
enemyType + "...");

                    System.out.println("\nGame Over");
```

```
                System.out.print("\nContinue adventuring? (Y for yes, Type anything for no): ");

                String ret = input.nextLine();

                if (ret.equalsIgnoreCase("Y")) {

                    retry = true;

                    isBattleOn = false;

                } else {

                    retry = false;

                    isBattleOn = false;

                }

                break;

            }

            break;



        case 2:

            System.out.println("\nChoose an item:");

            System.out.println("1. Health Potion");

            System.out.println("2. Sword");

            System.out.print("Enter your choice: ");

            int itemChoice = input.nextInt();

            input.nextLine();  // Consume the leftover newline
```

```
                if (itemChoice == 1) {
                    healthPotion.use(player);
                    System.out.println(playerName + " used a Health Potion
and recovered health!");
                } else if (itemChoice == 2) {
                    sword.use(player);
                    System.out.println(playerName + " used a Sword to
boost attack power!");
                } else {
                    System.out.println("Invalid choice. Returning to main
menu.");
                }


                if (enemy.isAlive()) {
                    if (rand.nextBoolean()) {
                        if (rand.nextBoolean()) {
                            poison.use(player, enemy);
                            System.out.println("The " + enemyType + " used
Poison! You took damage and your attack power is reduced.");
                        } else {
                            magicOrb.use(player, enemy);
```

```
                    System.out.println("The " + enemyType + " used
Magic Orb! Its attack power is now greatly increased.");

                }

            } else {

                enemy.attack(player);

                System.out.println("The " + enemyType + " attacks you
for " + enemy.getAttackPower() + " damage!");

            }

        }



        if (!player.isAlive()) {

            System.out.println("\nYou were defeated by the " +
enemyType + "...");

            System.out.print("\nContinue adventuring? (Y for yes,
Type anything for no): ");

            String ret = input.nextLine();

            if (ret.equalsIgnoreCase("Y")) {

                retry = true;

                isBattleOn = false;

            } else {

                retry = false;

                isBattleOn = false;

            }
```

```
                break;
            }
            break;

        case 3:
            System.out.println("\nYou ran away! The battle is over.");
            System.out.print("\nContinue adventuring? (Y for yes, Type anything for no): ");
            String ret = input.nextLine();
            if (ret.equalsIgnoreCase("Y")) {
                retry = true;
                isBattleOn = false;
            } else {
                retry = false;
                isBattleOn = false;
            }
            break;

        case 4:
            System.out.println("\nChoose which combat log to view:");
            System.out.println("1. Player's Combat Log");
```

```
                System.out.println("2. Enemy's Combat Log");

                System.out.print("Enter your choice: ");

                int logChoice = input.nextInt();

                input.nextLine();



                if (logChoice == 1) {

                  System.out.println(player.toString());

                } else if (logChoice == 2) {

                  System.out.println(enemy.toString());

                } else {

                  System.out.println("Invalid choice. Returning to main
menu.");

                }

                break;



              default:

                System.out.println("Invalid choice. Try again.");

                break;

            }

          }

        }
```

```
        input.close();
        System.out.println("\nThanks for playing!");
    }
}
```

Player Class:

```
public class Player {
    private String name;
    private int health;
    private int attackPower;

    public Player(String name) {
        this.name = name;
        this.health = 100;
        this.attackPower = 10;
    }

    public int attack(Enemy enemy) {
        enemy.takeDamage(attackPower);
        return attackPower;
```

```
}

public void takeDamage(int damage) {
    this.health -= damage;
}

public void increaseHealth(int amount) {
    this.health += amount;
    if (this.health > 100) this.health = 100;
}

public void increaseAttackPower(int amount) {
    this.attackPower += amount;
}

public boolean isAlive() {
    return this.health > 0;
}

public String toString() {
```

```
        return name + " (Health: " + health + ", Attack Power: " +
attackPower + ")";

    }



    public int getAttackPower() {

        return attackPower;

    }



    public void setAttackPower(int attackPower) {

        this.attackPower = attackPower;

    }

}
```

Black Magic Class:

```
public class BlackMagic {
    private String name;

    private int effect;



    public BlackMagic(String name, int effect) {

        this.name = name;

        this.effect = effect;

    }
```

```
    public int use(Player player, Enemy enemy) {

        if (name.equals("Poison")) {

            player.takeDamage(effect);

            player.setAttackPower(player.getAttackPower() -
(player.getAttackPower() / 10));

        } else if (name.equals("Magic Orb")) {

            enemy.takeDamage(-enemy.getAttackPower() * 2);

        }

        return effect;

    }



    public String toString() {

        return name + " (Effect: " + effect + ")";

    }
}
```

Item Class:

```
public class Item {
    private String name;

    private int effect;



    public Item(String name, int effect) {
```

```
        this.name = name;
        this.effect = effect;
    }

    public int use(Player player) {
        if (name.equals("Health Potion")) {
            player.increaseHealth(effect);
        } else if (name.equals("Sword")) {
            player.increaseAttackPower(effect);
        }
        return effect;
    }

    public String toString() {
        return name + " (Effect: " + effect + ")";
    }
}
```

Enemy Class:

```
public class Enemy {
    private String type;
```

```
private int health;

private int attackPower;


public Enemy(String type) {

   this.type = type;

   this.health = 50;

   this.attackPower = 5;

}


public void attack(Player player) {

   player.takeDamage(attackPower);

}


public void takeDamage(int damage) {

   this.health -= damage;

}


public boolean isAlive() {

   return this.health > 0;

}
```

```
    public int getAttackPower() {
        return this.attackPower;
    }

    public String toString() {
        return type + " (Health: " + health + ", Attack Power: " + attackPower
+ ")";
    }
}
```

Question 5

Section	**Description**
1. Problem	Complete the Java code that manages a collection of movies.
2. Solution	Follow the criteria needed by the questions to create classes to manage movie collection: 1. Add movie to a list 2. Remove movie from list by its title 3. Find movies based on the director 4. Find movie by its title 5. Sort movie in ascending order based on year 6. Find movie in a range of year of release 7. Display all movie in list

3. Sample Input & Output	`Searching for a movie by title:`
	`Movie 'Inception' found:`
	`- Title: Inception`
	`  Director: Christopher Nolan`
	`  Type: Science Fiction`
	`  Year: 2010`
	
	`Sorted movieCollection by release year:`
	`- Title: The Godfather`
	`  Director: Francis Ford Coppola`
	`  Type: Crime`
	`  Year: 1972`
	
	`- Title: The Shawshank Redemption`
	`  Director: Frank Darabont`
	`  Type: Drama`
	`  Year: 1994`
	
	`- Title: Pulp Fiction`
	`  Director: Quentin Tarantino`
	`  Type: Crime`
	`  Year: 1994`
	
	`- Title: The Dark Knight`
	`  Director: Christopher Nolan`
	`  Type: Action`
	`  Year: 2008`
	
	`- Title: Inception`
	`  Director: Christopher Nolan`
	`  Type: Science Fiction`
	`  Year: 2010`

<table>
<tr>
<td></td>
<td>

```
Removed movie: Pulp Fiction

Updated Movie Collection:
- Title: The Godfather
  Director: Francis Ford Coppola
  Type: Crime
  Year: 1972

- Title: The Shawshank Redemption
  Director: Frank Darabont
  Type: Drama
  Year: 1994

- Title: The Dark Knight
  Director: Christopher Nolan
  Type: Action
  Year: 2008

- Title: Inception
  Director: Christopher Nolan
  Type: Science Fiction
  Year: 2010
Searching for movie(s) by director:
Movies directed by 'Christopher Nolan'
- Title: Inception
  Director: Christopher Nolan
  Type: Science Fiction
  Year: 2010

- Title: The Dark Knight
  Director: Christopher Nolan
  Type: Action
  Year: 2008
```

</td>
</tr>
<tr>
<td>4. Source Code</td>
<td>

link

```
import java.util.*;

public class VIVA3 {

  public static class Movie{

    private String title;
    private String director;
```

</td>
</tr>
</table>

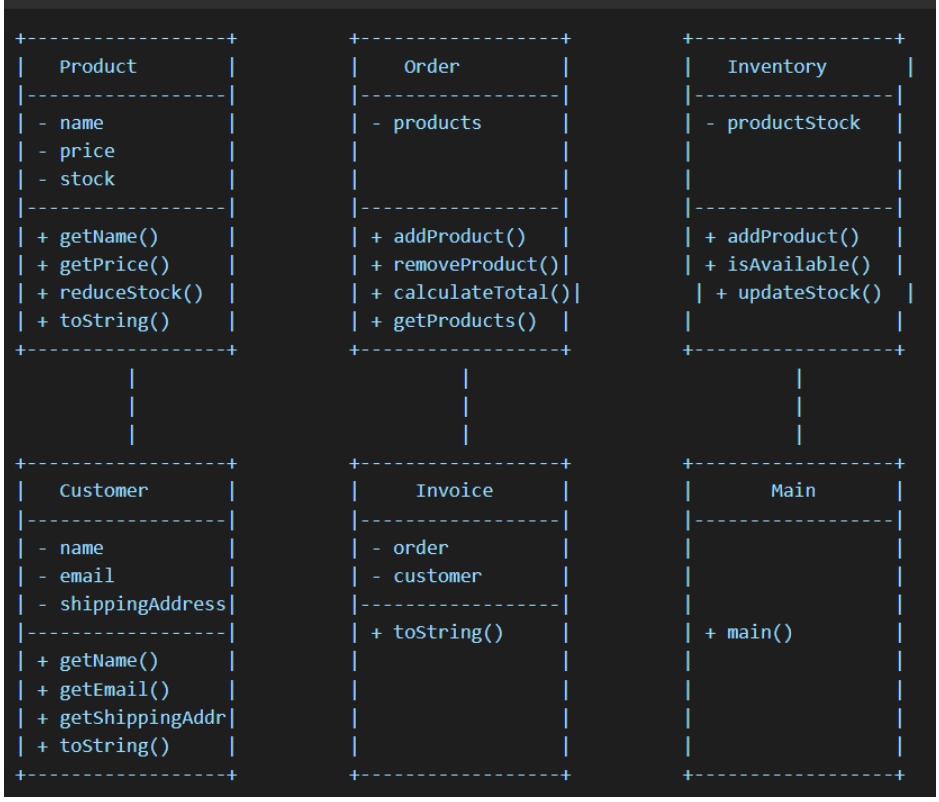
```
    private int year;
    private String type;

    public Movie(String title, String director, int year, String type){
        this.title= title;
        this.director= director;
        this.year= year;
        this.type= type;
    }

    public String getTitle(){
        return title;
    }
    public String getDirector(){
        return director;
    }
    public int getYear(){
        return year;
    }
    public String getType(){
        return type;
    }

    @Override
    public String toString(){
        return "- Title: " + title + "\n" +
               "  Director: " + director + "\n" +
               "  Type: " + type + "\n" +
               "  Year: " + year + "\n";
    }
}

public static class MovieCollection{
    private List<Movie> movieCollection;
    private int numMovie;

    public MovieCollection(){
        movieCollection= new ArrayList<>(100);
        numMovie=0;
    }
```

```
public boolean isEmpty(){
   return movieCollection.isEmpty();
}

public void addMovie(Movie movie){
   if(numMovie<100){
      movieCollection.add(movie);
      numMovie++;
   }
   else{
      System.out.println("The collection of movie is full");
   }
}

public void removeMovie(String title){
   Movie delete=null;

   for(Movie movie: movieCollection){
      if (movie.getTitle().equals(title)){
         delete=movie;
         break;
      }
   }

   if(delete!=null){
      System.out.println("Removed movie: "+ title);
      movieCollection.remove(delete);
      numMovie--;
   }
   else{
      System.out.println("Movie is not in the collection");
   }
}

public void findmovieCollectionByDirector(String director){
   boolean found = false;
   System.out.println("Movies directed by '" + director + "'");
   for(Movie movie: movieCollection){
      if (movie.getDirector().equals(director)){
         System.out.println(movie.toString());
         found=true;
```

```
            }
        }
        if(found==false){
            System.out.println("No movie form this director inside the
collection");
        }
    }

    public void findMovieByTitle(String title){
        System.out.println("Movie '" + title + "' found:");
        for(Movie movie: movieCollection){
            if (movie.getTitle().equals(title)){
                System.out.println(movie.toString());
            }
        }
    }

    public void sortmovieCollectionByReleaseYear(){
        Collections.sort(movieCollection,
Comparator.comparingInt(Movie ::getYear));
    }

    public void findmovieCollectionByReleaseYearRange(int startYear,
int endYear){
        for(Movie movie: movieCollection){
            if (movie.getYear()>=startYear && movie.getYear()<=endYear){
                System.out.println(movie.toString());
            }
        }
    }

    public void displayCollection(){
        for(Movie movie: movieCollection){
                System.out.println(movie.toString());
        }
    }
  }

  public static void main(String[] args) {
    MovieCollection movieCollection = new MovieCollection();
```

```
    Movie movie1 = new Movie("Inception", "Christopher Nolan",
2010,"Science Fiction");
    Movie movie2 = new Movie("The Shawshank Redemption", "Frank
Darabont", 1994, "Drama");
    Movie movie3 = new Movie("Pulp Fiction", "Quentin Tarantino", 1994,
"Crime");
    Movie movie4 = new Movie("The Dark Knight", "Christopher Nolan",
2008, "Action");
    Movie movie5 = new Movie("The Godfather", "Francis Ford
Coppola", 1972, "Crime");

    movieCollection.addMovie(movie1);
    movieCollection.addMovie(movie2);
    movieCollection.addMovie(movie3);
    movieCollection.addMovie(movie4);
    movieCollection.addMovie(movie5);

    System.out.println("Searching for movie(s) by director:");
    movieCollection.findmovieCollectionByDirector("Christopher
Nolan");

    System.out.println("\nSearching for a movie by title:");
    movieCollection.findMovieByTitle("Inception");

    System.out.println("\nMovies released between 1990 and 2010:");
    movieCollection.findmovieCollectionByReleaseYearRange(1990,
2010);

    System.out.println("\nSorted movieCollection by release year:");
    movieCollection.sortmovieCollectionByReleaseYear();

    movieCollection.displayCollection();

    movieCollection.removeMovie(movie3.getTitle());

    System.out.println("\nUpdated Movie Collection:");
    movieCollection.displayCollection();
  }

}
```

Question 6

Section	Description
1. Problem	The task is to create an order fulfilment system with the following key components: 1. **Add products to inventory** 2. **Remove products from inventory** 3. **Calculate the total payment of orders** 4. **Track the customer's order** 5. **Generate invoices for completed orders** The system should consist of five Java classes: `Product`, `Order`, `Customer`, `Inventory`, and `Invoice`, along with a `Main` class to test the functionality.
2. Solution	Class Diagram 
3. Sample Input & Output	

Output:

```
Power Bank is out of stock.
Invoice for John Doe
Email: john@example .com
Shipping Address: 1234 Main St

Products:
- Laptop: RM1200.0
- Smartphone: RM800.0

Total: RM2000.0


Process finished with exit code 0
```

4. Code

```java
public class Product {
    private final String name;
    private final double price;
    private int stock;

    // Constructor to initialize product attributes
    public Product(String name, double price, int stock) {
        this.name = name;
        this.price = price;
        this.stock = stock;
    }

    // Getter method to retrieve product name
    public String getName() {
        return name;
    }

    // Getter method to retrieve product price
    public double getPrice() {
        return price;
    }

    // Method to reduce stock by a specified quantity
    public void reduceStock(int quantity) {
        if (quantity > stock) {
            throw new IllegalArgumentException(name + " is out of
stock");
        }
        stock -= quantity;
```

```java
    }

    // Override the toString() method to return product details as a
string
    @Override
    public String toString() {
        return "Product{" +
                "name='" + name + ": " + '\'' +
                ", price=" + price +
                ", stock=" + stock +
                '}';
    }
}
import java.util.ArrayList;
import java.util.List;

// Order class represents a customer's order, which can contain
multiple products
public class Order {
    private final List<Product> products; // List to store products
added to the order

    // Constructor to initialize an empty list of products for the
order
    public Order() {
        this.products = new ArrayList<>();
    }

    // Getter method to retrieve the list of products in the order
    public List<Product> getProducts() {
        return products;
    }

    // Method to add a product to the order
    public void addProduct(Product product) {
        products.add(product);
        // System.out.println("Product added to the order: " +
product.getName());
    }

    // Method to remove a product from the order
    public void removeProduct(Product product) {
        products.remove(product);
        // System.out.println("Product removed from the order: " +
product.getName());
    }

    // Method to calculate the total price of all products in the order
    public double calculateTotal() {
        double total = 0;
        // Loop through each product and add its price to the total
```

```java
        for (Product product : products) {
            total += product.getPrice();
        }
          // System.out.println("Total price of the order: " + total);
        return total;
    }
}
// Customer class represents a customer placing an order

public class Customer {
    private final String name;
    private final String email;
    private final String shippingAddress;

    // Constructor to initialize the customer with their name, email,
and shipping address
    public Customer(String name, String email, String shippingAddress)
{
        this.name = name;
        this.email = email;
        this.shippingAddress = shippingAddress;
    }

    // Getter method to retrieve the customer's name
    public String getName() {
        return name;
    }

    // Getter method to retrieve the customer's email
    public String getEmail() {
        return email;
    }

    // Getter method to retrieve the customer's shipping address
    public String getShippingAddress() {
        return shippingAddress;
    }

    // Method to return a string representation of the customer object
    @Override
    public String toString() {
        return "Customer{" +
                "name='" + name + '\'' +
                ", email='" + email + '\'' +
                ", shippingAddress='" + shippingAddress + '\'' +
                '}';
    }
}
import java.util.Map;
import java.util.HashMap;
```

```java
public class Inventory {
    // A map to store the stock of products, with Product as the key
and the quantity as the value
    private final Map<Product, Integer> productStock;

    // Constructor to initialize the product stock map
    public Inventory() {
        this.productStock = new HashMap<>();
    }

    // Method to add a product and its quantity to the inventory
    public void addProduct(Product product, int qty) {
        productStock.put(product, qty);
    }

    // Method to check if the specified quantity of a product is
available in stock
    public boolean isAvailable(Product product, int qty) {
        return productStock.getOrDefault(product, 0) >= qty;
    }

    // Method to update the stock after selling or using the product
    public void updateStock(Product product, int qty) {
        product.reduceStock(qty);
        productStock.put(product, productStock.get(product) - qty);
    }
}
public class Invoice {
    private final Order order;
    private final Customer customer;

    // Constructor to initialize the invoice with an order and a
customer
    public Invoice(Order order, Customer customer) {
        this.order = order;
        this.customer = customer;
    }

    // Method to generate a string representation of the invoice
    @Override
    public String toString() {

        // Using StringBuilder for efficient string concatenation
        StringBuilder sb = new StringBuilder();
        // Adding customer details to the invoice
        sb.append("Invoice for
").append(customer.getName()).append("\n");
        sb.append("Email: ").append(customer.getEmail()).append("\n");
        sb.append("Shipping Address:
").append(customer.getShippingAddress()).append("\n");
        // Adding a section for products
```

```
        sb.append("\n");
        sb.append("Products:").append("\n");

        // Iterating over the products in the order and appending each
to the invoice
        for (Product product : order.getProducts()) {
            sb.append("- ").append(product.getName()).append(":
RM").append(product.getPrice()).append("\n");
        }
        // Adding the total price of the order
        sb.append("\n");
        sb.append("Total:
RM").append(order.calculateTotal()).append("\n");
        // Returning the final invoice string
        return sb.toString();
    }
}
public class Main {
    public static void main(String[] args) {

        // Creating instances of products with name, price, and initial
stock
        Product laptop = new Product("Laptop", 1200.00, 10);
        Product phone = new Product("Smartphone", 800.00, 5);
        Product powerBank = new Product("Power Bank", 100.00, 0);

        // Creating a customer with their name, email, and shipping
address
        Customer customer = new Customer("John Doe", "john@example
.com",
                "1234 Main St");

        // Creating an object (instance) of the Inventory class to
manage the product stock
        Inventory inventory = new Inventory();
        // Adding products to the inventory with specified quantities
        inventory.addProduct(laptop, 10);
        inventory.addProduct(phone, 5);
        inventory.addProduct(powerBank, 0);

        // Creating an object (instance) of the Order class and
assigning it to the 'order' variable
        Order order = new Order();

        // Checking if the laptop is available, adding it to the order
if available
        if (inventory.isAvailable(laptop, 1)) {
            order.addProduct(laptop);
            inventory.updateStock(laptop, 1);
        } else {
            System.out.println(laptop.getName() + " is out of stock.");
        }
```

```java
        // Checking if 2 smartphones are available, adding them to the
order if available
        if (inventory.isAvailable(phone, 2)) {
            order.addProduct(phone);
            inventory.updateStock(phone, 2);
        } else {
            System.out.println(phone.getName() + " is out of stock.");
        }

        // Checking if the power bank is available, and notifying if
out of stock
        if (inventory.isAvailable(powerBank, 1)) {
            order.addProduct(powerBank);
            inventory.updateStock(powerBank, 1);
        } else {
            System.out.println(powerBank.getName() + " is out of
stock.");
        }

        // Creating an invoice for the order and customer, then
printing it
        Invoice invoice = new Invoice(order, customer);
        System.out.println(invoice);
    }
}
```